

Radau: Numerical Method and Adaptive Execution

1 Differential equation and numerical method

Consider $z' = f(t, z)$, $z(t_0) = z_0$, in the project's packed physical layout. A three-stage Radau IIA collocation method of order five, with an embedded third-order error estimator and a cubic collocation interpolant. The implementation delegates the numerical tableau, local error estimator, trial rejection and interpolation polynomial to the installed SciPy **Radau** solver. The adapter does not duplicate these algorithms.

A Runge–Kutta evaluation has stage equations

$$Y_i = z_n + h \sum_j a_{ij} f(t_n + c_j h, Y_j), \quad z_{n+1} = z_n + h \sum_i b_i f(t_n + c_i h, Y_i).$$

The collocation equations are implicit and are solved by SciPy Newton iterations. The public adapter accepts an optional full-state Jacobian callable; otherwise SciPy uses finite differences. With energy tracking, a supplied Jacobian must cover all augmented coordinates.

2 Error control and accepted intervals

The adaptive backend scales local error estimates using positive absolute and relative tolerances, with component scale based on $\text{atol} + \text{rtol} \max(|z_n|, |z_{n+1}|)$. It selects subsequent step sizes and can reject trial steps. **max_step** bounds accepted duration; **first_step=None** delegates initial selection to the backend. Requested tolerances bound neither global trajectory error nor roundoff.

One live solver is retained over the complete interval. Its state, stage work and any Jacobian/factorization reuse survive accepted steps. The adapter calls **solver.step()** until acceptance or failure; rejected trials never become observer records or separate accepted metric rows.

3 Dense output and energy extension

Saved output times do not determine the accepted grid. Each requested sample is evaluated from the dense interpolant of its accepted interval. This does not restart the solver. **dense_output=True** explicitly constructs every accepted interpolant; optional observers can retain them for later time queries.

For a suitable time-dependent Hamiltonian system, optional tracking augments z with one momentum per particle:

$$k' = -\partial_t H(t, z), \quad k(t_0) = 0.$$

The exported generalized-energy diagnostic measures $H(t, z) + k$. Error control acts on the augmented vector, so this option can change the accepted time grid and the physical floating-point trajectory. It is not an independent proof of trajectory accuracy.

4 Shared architecture and observation

The public class owns initialization, advance, observation and history export. Inherited `new_run` constructs a fresh instance of the same class and calls `initialize`, which creates its live solver. There is no separate method context or record of callbacks. All 13 methods inherit the same integration entry point and use the common global coordinator.

`ScipyAdaptiveController` exposes accepted intervals from the retained numerical session. The common collector stores requested samples and small accepted-step statistics; it does not retain interpolants or events by default.

An `AdaptiveIntegrationStep` records times and copied states. It also exposes work counters and `dense_state`. It does not claim a fixed-duration candidate-state map: adaptive decisions depend on numerical history. `AdaptiveTrajectoryObserver` explicitly retains continuous output outside the integrator. The observer and any persistence belong to the caller.

Function evaluations, Jacobian evaluations and LU decompositions are counted per accepted interval. Initialization work belongs to the first row; rejected trial work belongs to the acceptance that follows it. Interpolation needed only for an observer is excluded from main counters unless continuous output was explicitly requested. The ordinary method operation `advance` accepts one step on the retained solver, taking an upper step bound. The controller reports the actual accepted endpoint. A run instance belongs to one execution; repeating `simulate` creates a fresh instance and solver.

5 Implementation and verification

The adapter is in `src/simulation/methods/adaptive/scipy.py`; shared coordination and collection are in `src/simulation/integration.py`. The model's six-phase diagram is documented in its `simulation/` directory. Tests compare sampled states and work counters with the same SciPy solver, check a known non-autonomous rotation, and verify optional energy, dense retention, repeated runs, observation isolation and failure handling. Scientific reference trajectories still require refinement and an independent audit; arithmetic precision and integration error remain separate.